Practical : 3	Implementation of max-heap sort algorithm
Date : 05/08/2026	

Aim: To implement and analyze the **Heap Sort algorithm using a Max-Heap**.

Objective :- To implement the Heap Sort algorithm using a Max-Heap structure in C++, and to evaluate its time and space complexities.

MAX-HEAP SORT Algorithm

1. Start.
2. Read the array and size n.
3. Build a Max-Heap.
4. Start from the last non-leaf node.
5. Compare the node with its left and right children.
6. Find the largest element.
7. If required, swap the node with the largest child.
8. Apply heapify recursively.
9. Swap the root element with the last element.
10. Reduce the heap size.
11. Apply heapify again.
12. Repeat until all elements are sorted.
13. Display the sorted array.
14. Stop.

Program:

```
#include <iostream>
using namespace std;

// Function to print array elements
void printArray(int arr[], int size) {
for (int i = 0; i < size; i++) {
cout << arr[i] << " ";
}

cout << endl;
}
// Function to heapify a subtree
```

```
void heapify(int arr[], int n, int i) {

int largest = i;

int left = 2 * i + 1;
int right = 2 * i + 2;

// If left child is larger than root
if (left < n && arr[left] > arr[largest]) {
largest = left;
}

// If right child is larger
if (right < n && arr[right] > arr[largest]) {
largest = right;
}

// If largest is not root
if (largest != i) {

int temp = arr[i];
arr[i] = arr[largest];
arr[largest] = temp;

// Heapify affected subtree
heapify(arr, n, largest);
}
}

// Function to perform Heap Sort
void heapSort(int arr[], int n) {

// Step 1: Build Max-Heap
for (int i = n / 2 - 1; i >= 0; i--) {
heapify(arr, n, i);
}


// Step 2: Extract elements
```

```
for (int i = n - 1; i > 0; i--) {

// Move root to the end
int temp = arr[0];
arr[0] = arr[i];
arr[i] = temp;

// Heapify reduced heap
heapify(arr, i, 0);
}
}

int main() {

int arr[] = {45, 12, 85, 32, 7, 90, 60};

int n = sizeof(arr) / sizeof(arr[0]);

cout << "Original Array: ";
printArray(arr, n);

heapSort(arr, n);

cout << "Sorted Array (Heap Sort): ";
printArray(arr, n);

return 0;
}
```

Output:

```
Output
Original Array: 45 12 85 32 7 90 60
Sorted Array (Heap Sort): 7 12 32 45 60 85 90


=== Code Execution Successful ===
```

Complexity Analysis:-

A. Time Complexity Analysis

Heap Sort operates in two distinct phases: building the initial max-heap and repeatedly extracting the maximum element.

- **Building the Max-Heap:** Converting an unsorted array of size n into a valid max-heap takes **O(n)** time.
- **Heapify during Extraction:** Extracting the root element and pushing it to the back requires running the heapify function. Since a binary heap has a max height of log n, each extraction swap costs O(log n) time. Doing this for all n elements takes **O(n log n)** time.

Combining these two phases gives Heap Sort a remarkably consistent time profile:

- **Best Case:** O(n log n)
- **Average Case:** O(n log n)
- **Worst Case:** O(n log n)

B. Space Complexity Analysis

- **Space Complexity: O(1) (Constant Space)**

  The sorting process happens completely in-place. The elements are rearranged within the original input array itself, requiring no external lists or auxiliary memory allocation.

Conclusion :-

Through this practical, we successfully implemented Heap Sort using a Max-Heap structure. The primary advantage of Heap Sort is its absolute dependability—it guarantees a highly efficient **O(n log n)** time complexity across all input conditions (best, worst, and average cases) while matching the in-place **O(1)** space efficiency of simpler quadratic sorts.